Features

În ordinea din PLAN.pdf · Rareș

□ bifa: feature-ul trece „Gata când” din PLAN.pdf · NEW = ceva ce nu ai făcut încă; înainte de feature, Bogdan îți predă lecția

Reguli pentru toate feature-urile

- Fiecare feature merge de la A la Z și aduce cel mult o unealtă nouă (F5, plecarea pe EC2, are două lecții înainte); niciun feature nu începe înainte de lecția lui.
- F1–F2 sunt fără login; securitatea și testele cu login real încep la F3, pe H2. Docker, EC2 și Testcontainers vin abia la F5.
- „Gata când”: F1–F4 local (de la F3 cu token), de la F5 pe EC2 prin https, cu token; partea III local sau pe instanța de demo.
- Erorile au forma `ApiError {timestamp, status, error, message, path, details[]}`.
- F-urile merg în ordine; fiecare U pornește doar după feature-ul scris la el, iar U5 se face între F14 și F15.

I. Salonul merge (monolit)

F1 Salonul se configurează și programările se gestionează din API

fără login încă · local · de la F3: ADMIN · BARBER · CUSTOMER

□ Administratorul adaugă, modifică și dezactivează frizeri; frizerul își pune procedurile cu prețul lui, programul săptămânal și perioadele libere; programarea se poate reprograma, confirma, anula și finaliza din HTTP.

nimic nou, doar ce știi deja · **Lecția:** niciuna

F2 Baza are istoric de schemă NEW

fără login încă · local

□ Schema bazei e scrisă de mână și versionată: fiecare schimbare e o migrare, iar aplicația refuză să pornească dacă tabelele nu se potrivesc cu entitățile. Parola și adresa bazei ies din repo; testele rămân pe H2.

NEW Flyway, configurarea din variabile de mediu · **Lecția:** L1 „Baza versionată”

F3 Contul meu: client, frizer, administrator NEW

vizitator · CUSTOMER · BARBER · ADMIN · local

□ Vizitatorul își face cont de client și se loghează; clientul își rezervă și își vede programările, frizerul își confirmă ziua, administratorul face conturile de frizer și blochează clienții. Fiecare vede și schimbă doar ce e al lui.

NEW Spring Security + JWT · **Lecția:** L2 „Spring Security cu JWT”

F4 Programarea mea recurentă: la 2 săptămâni, aceeași oră NEW

CUSTOMER · ADMIN · jobul de noapte · local

□ Clientul își face o serie; în fiecare noapte un job generează programările seriei pe următoarele 28 de zile, sare peste cele care nu încap și spune de ce.

NEW logare structurată (MDC cu id-ul rulării, loguri JSON) · **Lecția:** L3 „Joburi programate și loguri care se pot căuta”

F5 Salonul pe EC2, cu teste pe MySQL adevărat NEW

CUSTOMER · ADMIN · pe EC2

□ Aplicația rulează în Docker pe EC2, prin https pe domeniul tău; fiecare push pe main o testează pe MySQL adevărat și o livrează singur, iar două rezervări în aceeași secundă pe același slot nu mai pot trece amândouă.

NEW Docker + compose + GitHub Actions + EC2 (nginx + certbot) · Testcontainers + failsafe · `@Lock` + `READ COMMITTED` · **Lecțiile:** L4 „Docker și EC2”, apoi L5 „Două cereri în același timp”

F6 Ziua salonului, live NEW

ADMIN (în Grafana)

□ Administratorul vede în Grafana, în timp real, cât de ocupat e fiecare frizer azi, câte programări sunt pe fiecare status, câte anulări târzii au fost și dacă jobul de noapte a mers.

NEW Micrometer + Prometheus + Grafana în compose · **Lecția:** L6 „Metrici cu Micrometer și Grafana”

II. MCP — piesa centrală

F7 Salonul văzut de un agent NEW

orice client MCP cu token (BARBER, ADMIN, apoi AGENT)

- [ ] Un client MCP se conectează cu un token și întreabă de frizeri, proceduri, program, time-off, sloturi libere, programări și tiparul de vizite al unui client; vede exact cât vede utilizatorul tokenului.

NEW server MCP (Spring AI 2.0.2) · **Lecția:** L7 „Server MCP cu Spring AI”

F8 Frizerul lipsește: propun → clientul aprobă → se reprogramează NEW

ADMIN · AGENT · CUSTOMER

- [ ] Administratorul pune un time-off urgent, agentul propune o oră nouă pentru fiecare programare rămasă, iar clientul acceptă sau refuză; acceptarea mută programarea pe aceeași cale ca o reprogramare făcută de mână.

NEW tool-uri MCP de scriere (propunere → aprobare de un om → aceeași cale de execuție) · **Lecția:** L8 „Agentul propune, omul decide”

F9 Asistentul de reprogramare pe reguli — primul microserviciu NEW

AGENT

- [ ] O aplicație separată, booking-agent, verifică la fiecare 5 minute prin MCP ce programări au rămas fără frizer, aplică regulile și propune o oră nouă pentru fiecare.

NEW client MCP într-o a doua aplicație · **Lecția:** L9 „Al doilea proiect Spring: client MCP”

F10 Asistentul cu AI: ora potrivită pentru client NEW

AGENT

- [ ] Același agent primește un al doilea planner, cu Claude, care alege ora la care clientul vine de obicei, cu un mesaj scris pentru el; din configurare alegi care planner rulează.

NEW Spring AI ChatClient cu Claude · **Lecția:** L10 „Spring AI ChatClient cu Claude”

III. Evenimente și servicii

F11 Programarea anunță, ceilalți reacționează NEW

CUSTOMER · AGENT

- [ ] Fiecare schimbare a unei programări și fiecare propunere devin evenimente; clientul primește o notificare, iar agentul pornește imediat ce o programare rămâne fără frizer, nu la 5 minute.

NEW Kafka · **Lecția:** L11 „Kafka în monolit”

F12 Notificările pleacă din monolit NEW

CUSTOMER

- [ ] Notificările clienților devin serviciul lor, booking-notifications, cu baza lor; între el și API singurul contract sunt topicurile.

NEW al doilea serviciu desprins, topicurile = contractul · **Lecția:** L12 „Un serviciu desprins”

F13 Urmăresc o reprogramare prin tot sistemul NEW

ADMIN (în Grafana și Kibana)

- [ ] De la time-off-ul urgent până la notificarea clientului, o singură reprogramare se vede ca un drum: API → Kafka → agent → MCP → Claude → propunere → notificare → acceptare.

NEW OpenTelemetry + Tempo + ELK (Elasticsearch, Kibana, Filebeat) · **Lecția:** L13 „Tracing și loguri centrale”

IV. Platforma: login central și o singură poartă

F14 Cine ești, spus de Keycloak NEW

toți · AGENT · ADMIN

- [ ] Conturile și parolele nu mai stau în API: Keycloak emite tokenurile pentru clienți, frizeri și admin, pentru agent fără parolă și pentru UI-urile de administrare.

NEW Keycloak (OAuth2 / OIDC) · **Lecția:** L14 „Keycloak”

F15 O singură poartă: Kong NEW

toți

- [ ] Aplicațiile nu mai sunt expuse direct: tot traficul, inclusiv al agentului, trece printr-un API gateway care verifică tokenul, limitează abuzul și pune un id pe fiecare cerere.

NEW Kong (API gateway DB-less) + rețele Docker · **Lecția:** L15 „Kong și rețelele Docker”

V. Interfața — React (booking-client)

U1 Îmi aleg ora din browser NEW

client, fără login încă · după F1 și react-api-calls · local

- [ ] Pe o pagină: alegi frizerul, bifezi procedurile, alegi ziua, vezi sloturile libere și rezervi cu un click; dacă slotul s-a ocupat între timp, vezi mesajul.

NEW date de la server în React · **Lecția:** R1 = `react-api-calls` din cursul de React (după `user-management-local`); U1 = aplicarea ei, ca `user-management-api`

U2 Contul meu și programările mele NEW

CUSTOMER · BARBER · după F3 · local

- [ ] Te loghezi; clientul își vede și își anulează programările, frizerul își vede ziua și confirmă / finalizează.

NEW componente cu props · **Lecția:** R2 „Componente cu props”

U3 Ziua salonului, pe pagini NEW

ADMIN · CUSTOMER · BARBER · după F6 · pe EC2

- [ ] Aplicația are pagini cu adrese: rezervare, programările mele, ziua frizerului și, pentru admin, ziua salonului cu ocuparea fiecărui frizer, actualizată singură.

NEW React Router · **Lecția:** R3 „Pagini cu React Router”

U4 Accept ora propusă NEW

CUSTOMER · ADMIN · după F8 (F9–F10 pentru date reale)

- [ ] Clientul vede propunerea de reprogramare cu mesajul pentru el și apasă Accept sau Refuz; adminul poate decide în numele clientului.

NEW test end-to-end Playwright · **Lecția:** R4 „Test e2e cu Playwright”

U5 Login prin Keycloak NEW

toți · imediat după F14, înainte de F15

- [ ] Formularul propriu de login dispare: te loghezi pe pagina Keycloak și te întorci în aplicație.

NEW OIDC: authorization code + PKCE · **Lecția:** R5 „Login OIDC cu PKCE”